

General Reminders

<code>#include "myfile.h"</code>	Include file.
<code>rand()</code>	Random int, <code>#include<cstdlib></code> .
<code>int(var)</code>	Convert a data type var to int.
<code>float(var)</code>	Convert a data type var to float.
<code>double(var)</code>	Convert a data type var to double.
<code>static_cast<t>(var)</code>	Convert var to the type t.
<code>void myF() const</code>	read only fonction.
<code>inline</code>	the whole code of the inline function is inserted or substituted at the point of its call during the compilation.
<code>constexpr</code>	that specify that an expression must be evaluated at compile time.
<code>sizeof(var)</code>	return the number of bytes used by the variable. <code>sizeof</code> runs at compile-time.
<code>std::move(obj)</code>	During assignation move the already existing object instead of creating a copy of it (memory optimizations).
<code>typedef T A</code>	Create an alias that refers to the type T.
<code>ternary Operator</code>	condition ? ifTrue : ifFalse

Strings

<code>str[i]</code>	Get or set the char at the index i.
<code>.length()</code>	Return the number of characters.
<code>.substr(a,b)</code>	Returns the substring starting at index a with length b.
<code>.find(subStr)</code>	Return the start index of the substring
<code>.replace(i,l,str)</code>	Replace l characters starting at index i with str.
<code>#include <string></code>	all the above must include string.
<code>stoi(str)</code>	Convert a string to int.
<code>stof(str)</code>	Convert a string to float,
<code>stod(str)</code>	Convert a string to double,
<code>to_string(var)</code>	Convert var to a string,

Vectors

<code>#include<vector></code>	Include vector library.
<code>vector<type> V;</code>	Instantiate a vector.
<code>vector<type> V{6,3,3};</code>	Instantiate a vector from Array.
<code>vector<type> V(s, var);</code>	Instantiate a vector of size s with all elements initialized to var.
<code>v[i]</code>	Get or set the element at index i.
<code>.at(i)</code>	Returns the element at index i and performs a bound check.
<code>.size()</code>	Return the number of elements.
<code>.push_back(Value)</code>	Add the new element at the end.
<code>.pop_back()</code>	Remove the last element.
<code>.clear()</code>	Empty the vector.
<code>.insert(i, Value)</code>	Insert Value at i.
<code>.reserve(size)</code>	Pre-allocate memory for performance, Use it when the maximum size is known.
<code>.front()</code>	Returns the first element.
<code>.back()</code>	Returns the last element.
<code>.empty()</code>	Returns true if vector is empty.
<code>.erase(it)</code>	Erase element at iterator it.

Arrays

0	1	...	n	This table illustrate the structure of an array of strings. Considering that n is equal to the
"Max"	"Tom"	...	arr[n]	

number of element minus one. Arrays are a static data type.

<code>int arr[4];</code>	Create a array of int and with 4 element.
<code>int arr[4]={6,3};</code>	
<code>arr[i]</code>	Get or set the element at the index i.

Structures

```
struct myStruct {
    string param1;    // attribute 1
    double param2;    // attribute 2
}s1, s2;             // myStruct instances
```

<code>myStruct Obj;</code>	instantiate structure object.
<code>Obj.param1</code>	Access param1 of Obj.

Streams

<code>#include<fstream></code>	Include stream library.
<code>#include<sstream></code>	Include string stream library.
<code>ifstream fin;</code>	Instantiate a input stream.
<code>ofstream fout;</code>	Instantiate a output stream.
<code>fstream f(fileMode);</code>	Instantiate a input/output stream.
<code>stringstream s(str);</code>	Instantiate a string stream.
<code>fout<<"hello"</code>	Output in stream "helloWorld".
<code>fin>>var</code>	Input from stream to var.

fstream functions

<code>getline(fin, line)</code>	Get the next line from fin and assign it to line.
<code>.open("file.txt")</code>	Open txt file whith the stream.
<code>.is_open()</code>	Return true if the file is open.
<code>.close()</code>	Close the stream file.

File modes

<code>ios::in</code>	Open for reading.
<code>ios::out</code>	Open for writing.
<code>ios::app</code>	Open for appending (writing at end).
<code>ios::trunc</code>	Empty the file if it exists.
<code>ios::ate</code>	Open and start at the end of the file.
<code>ios::binary</code>	Open in binary mode (no text processing).

File modes can be combined using the bitwise OR operator |.

manipulators

<code><<dec<<</code>	Set number base to decimal.
<code><<hex<<</code>	Set number base to hexadecimal.
<code><<oct<<</code>	Set number base to octal.
<code><<scientific<<</code>	Display floating-point numbers in scientific notation.
<code><<setprecision(n)<<</code>	Set decimal points, <code>#include<iomanip></code> .
<code><<setw(n)<<</code>	Establishes a print field of n spaces.
<code><<fixed<<</code>	Display floating point numbers in fixed point notation.
<code><<showpoint<<</code>	Enables or disables the unconditional inclusion of the decimal point character in floating-point output.
<code><<left<<</code>	output the string on the left.
<code><<right<<</code>	output the string on the right.

clear buffer

The buffer must be cleared after after getting an input from a stream if you input and output in the same file at the same time.

```
if(cin.fail() == true) {
    cin.clear();
    cin.ignore(1000, '\n');
}
```

Error Handling

```
try {
    // risky operation
} catch (exceptions) {
    // runs if an exception of type Ex is thrown
}

#include<cassert>      Include assert library.
#include<stdexcept>    Common standard exceptions.
throw myException      Throw an error of type myException.
exception::what()       Retrieve diagnostic message.
catch (const auto& e)   Catch exceptions by const reference.
catch(...)              Fallback handler; rethrow if unsure.
exception               Parent of all exceptions class.
```

Object Oriented Programing(OOP)

```
class myClasses :public parentClass{
private:
    // private methods and variables
public:
    // public methods and variables

    myClasses(int p1, int p2){...} // Constructor

    ~myClasses(){...} // Destructor

    // Override the inherited method parentMethod()
    void parentMethod() override { ... }

    //Example Operator Overloading
    Number operator+(const Number &n){
        return Number(value + n.value);
    }
};

myClasses myObj(3,5);    Instantiate an myClasses type obj.
myClasses myObj;         Call the default constructor.
protected:              similar to private, but it can also be
                          accessed in the inherited class.

virtual                  Specify that a method can be overridden
                          in a derived class.
```

OOP With header file

If you use a header the file wich contain the main function must include the header file.

Header file(myHeader.h)

```
#ifndef MYCLASS_H //if no def for MyClass
#define MYCLASS_H //else

using namespace std;

class MyClass{
public:
    :
private:
    :
};
#endif
```

Class file(.cpp)

```
#include "myHeader.h"

MyClass::MyClass(int p1, ...){
    publicAttribute = p1;
    :
}

}
```

Genericity

```
template <typename T_1, ..., typename T_n>
class myClasse{
    :
}
```

```
myClasse<T_1, ..., T_n>(...);
```

Switch case

```
switch (x){
    case 0:
        /*Code in case x = 0*/
        break;
        :
        :
    case n:
        /*Code in case x = n*/
        break;
    default:
        /*Code if no case match*/
}
```

Pointer & References

int* myInt;	* means myInt work form a pointer.
new	dynamically allocate a block of memory.
delete	release dynamically allocated memory.
NULL	Macro that referens to null pointer.
*var	Get var value, where var is a pointer.
&var	Get memory addresse of var .
void* var	Pointer with no associated data type.
p->m	Access member m of the object pointed to by p .

Smart Pointers

#include<memory>	Include memory library.
unique_ptr<T> uptr	Unique ownership smart pointer, only one pointer can point at the variable.
make_unique<T>(args)	Create a unique_ptr instance.
shared_ptr<T> sptr	Shared ownership smart pointer, multiple pointers can point at the variable.
make_shared<T>(args)	Create a shared_ptr instance.
.use_count()	Get the number of shared_ptr instances managing the same object.
weak_ptr<T> wptr	Non-owning smart pointer (observe only), used to bre circular references with shared_ptr.
.lock()	Get a shared_ptr from a weak_ptr. Returns an empty shared_ptr if the managed object has been destroyed.
wptr.expired()	Check if the managed object has been destroyed.

Bitwise Operators

&	Bitwise AND.	~	Bitwise NOT.	^	Bitwise XOR.
	Bitwise OR.	<<	Left shift.	>>	Right shift.

Namespaces

namespace NS {...}	Define a namespace.
NS::func()	Access member of namespace.
using namespace NS;	Import all names from namespace.
using NS::func;	Import specific name from NS.
namespace {...}	Anonymous namespace: limits scope to current translation unit (file).
inline namespace NS {...}	Members are accessible without qualification by default.
namespace alias = NS;	Create an alias for a namespace.
::globalVar	Access global namespace explicitly.

Lambda Expression

```
... = [captureClause] (parameters) -> returnType {
    // definition}

[&]      capture all external variables by reference.
[=]      capture all external variables by value.
[a,&b]    capture 'a' by value and 'b' by reference.
```

cmath

#include<cmath>	Include cmath library.
sqrt(x)	Square root of x .
pow(x, y)	x raised to the power y .
abs(x)	Absolute value overloads.
floor(x)	Greatest integer $\leq$ x .
ceil(x)	Smallest integer $\geq$ x .
fmod(x, y)	Floating-point remainder of x/y .

Advanced Memory Management

<code>malloc(size)</code>	Allocate <code>size</code> bytes of memory.
<code>calloc(n, size)</code>	Allocate memory for an array of <code>n</code> elements of <code>size</code> bytes each, initialized to zero.
<code>realloc(ptr, size)</code>	Reallocate memory to a new size.
<code>free(obj)</code>	dynamically de-allocate the memory allocated with <code>malloc</code> .

Random

<code>#include <random></code>	Include the standard random library.
<code>random_device</code>	
<code>mt19937</code>	Linear distributed random number generator object.
<code>uniform_real_distribution</code>	Uniform random number generator object.
<code>normal_distribution</code>	Normal random number generator object.

Special Ints

signed fixed width integer types			
<code>int8_t</code>	<code>int16_t</code>	<code>int32_t</code>	<code>int64_t</code>
<code>int_fast8_t</code>	<code>int_fast16_t</code>	<code>int_fast32_t</code>	<code>int_fast64_t</code>
<code>int_least8_t</code>	<code>int_least16_t</code>	<code>int_least32_t</code>	<code>int_least64_t</code>
unsigned fixed width integer types			
<code>uint8_t</code>	<code>uint16_t</code>	<code>uint32_t</code>	<code>uint64_t</code>
<code>uint_fast8_t</code>	<code>uint_fast16_t</code>	<code>uint_fast32_t</code>	<code>uint_fast64_t</code>
<code>uint_least8_t</code>	<code>uint_least16_t</code>	<code>uint_least32_t</code>	<code>uint_least64_t</code>
other integer types			
<code>intmax_t</code> & <code>uintmax_t</code>	Maximum-width integer type.		
<code>intptr_t</code> & <code>uintptr_t</code>	Integer types capable of storing a pointer value.		
<code>size_t</code>	An unsigned integer data type to represent the size of objects in bytes; commonly used for array indexing and loop counters.		

Preprocessing

<code>#define NAME value</code>	Define a macro.
<code>#define F(x) x*x</code>	Define a function-like macro.
<code>#ifdef NAME</code>	If the macro <code>NAME</code> is defined.
<code>#ifndef NAME</code>	If the macro <code>NAME</code> is not defined.
<code>#else</code>	Alternative case for <code>ifdef/ifndef</code> .
<code>#endif</code>	End conditional directive.
<code>#include</code>	Include a file.
<code>__FILE__</code>	Current file name.
<code>__LINE__</code>	Current line number.
<code>__DATE__</code>	Compilation date.
<code>__TIME__</code>	Compilation time.
<code>#pragma</code>	Implementation-specific instruction.

ple include guard for header files. ● `pack(push, n)` / `#pragma pack(pop)` — set and restore struct packing/alignment to `n` bytes. ● `pack(n)` — set struct member alignment to `n`. ● GCC `optimize("...")` — enable compiler-specific optimizations (GCC/Clang). ● `#pragma warning(push)` / `#pragma warning(pop)` / `#pragma warning(disable:NNNN)` — control MSVC warnings. ● `#pragma message("text")` — emit a compile-time message. ● `#pragma comment(lib, "name.lib")` — instruct MSVC linker to link a library.

Compiler Comments

<code>clang++ fileName</code>	comment to compile c++ code with clang , define the name of the compiled object.
<code>-o name</code>	Makes the compiler print detailed information. "v" stands for "Verbose".
<code>-v</code>	Prints the preprocessor output.
<code>-E</code>	generate an object file. To add .o file to the compilation simply add those like a regular file.
<code>-c fileName</code>	Optimizations levels, where <code>-O0</code> does not optimize the code.
<code>-O0, -O1, -O2, -O3, -Os</code>	Optimize for size.
<code>-fast-math</code>	Enable fast unsafe math optimizations.
<code>-march=native</code>	Optimize code for the architecture of the machine on which the compiler is running.
<code>-std=c++17</code>	Specify version to C++ 17.
<code>-Wall</code>	activates all default warnings
<code>-Wextra</code>	Enable extra warnings beyond <code>-Wall</code> .
<code>-Wpedantic</code>	Reject all programs that use forbidden extensions.
<code>-Werror</code>	Make all warnings into errors.
<code>-Wshadow</code>	Warn for variables that shadow other variables.
<code>-Wconversion</code>	Warn for implicit type conversions that may alter a value.
<code>-Wcast-qual</code>	Warn whenever a pointer is cast so as to remove a type qualifier from the target type.
<code>-Wstrict-prototypes</code>	Warn if a function is declared or defined without specifying the argument types.

Basic syntax

```
if (myBoolean){
    :
}

while(myBoolean){
    :
}

for (size_t i = 0; i < n; i++){
    :
}

for (auto Obj: Lst){
    :
}

void function (TYPE1 var, TYPE2 defaultVar = value){
    :
    return something;
}
```